

Task 7: OOPs – Inheritance and overriding

- Create a child class called **GraduateStudent** that extends your base Student. This allows the child class to automatically reuse the fields, getters, array logic, and validation rules from the parent class without rewriting a single line of that code.
- Add Unique Child Traits: We add a new private attribute unique to graduate students: private String thesisTitle, along with its own public getter (getThesisTitle()).
- introduce an status metric called **qualifiesForHonorRoll()** inside the parent class, which checks if a student has an average of **90.0% or higher**. Then, using **Method Overriding**, the GraduateStudent child class will rewrite this exact method to enforce a stricter rule: graduate students must maintain a **95.0% or higher** to make the honor roll.

```
// — MAIN RUNNER EXECUTIVE SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        System.out.println("=== Graduate Student Data Entry ===");
        System.out.print("Enter raw student name (e.g., ' mariah carey '): ");
        String rawName = inputScanner.nextLine();

        System.out.print("Enter Research Thesis Topic: ");
        String thesis = inputScanner.nextLine();

        // Child Object Birth
        GraduateStudent activeGrad = new GraduateStudent(rawName, thesis, inputScanner);

        // --- REPORT GENERATION ---
        System.out.println("\n=====");
        System.out.println("      GRADUATE ACADEMIC REPORT      ");
        System.out.println("=====");
        System.out.println("Student Profile: " + activeGrad.getName());
        System.out.println("Total Classes:   " + activeGrad.getGrades().length);
        System.out.println("Research Focus:  " + activeGrad.getThesisTitle());

        double avg = activeGrad.calculateAverage();
        System.out.println("Final Average:   " + avg + "%");

        // — EVALUATING THE OVERRIDDEN METHOD —
        // Java executes the customized child method logic rules automatically!
        if (activeGrad.qualifiesForHonorRoll()) {
            System.out.println("Honor Roll:      ★ APPROVED (95%+ Target Cleared) ★");
        } else {
            System.out.println("Honor Roll:      Standard Graduate Standing");
        }
        System.out.println("=====");

        inputScanner.close();
    }
}
```

```
// — INHERITANCE (CHILD CLASS) —
class GraduateStudent extends Student {

    private String thesisTitle;

    // Child Constructor calling parent super
    public GraduateStudent(String rawName, String thesisTitle, Scanner inputScanner) {
        super(rawName, inputScanner);
        this.thesisTitle = thesisTitle;
    }

    public String getThesisTitle() {
        return this.thesisTitle;
    }

    // — NEW STEP: METHOD OVERRIDING —
    @Override
    public boolean qualifiesForHonorRoll() {
        // Graduate students must hit a tougher threshold (95.0%)
        // Notice it calls the inherited calculateAverage() method seamlessly!
        return calculateAverage() >= 95.0;
    }
}
```

- **Parent Method:** We add public boolean qualifiesForHonorRoll() inside the Student class with a base threshold of >= 90.0.
- **Method Overriding:** We copy the exact same method signature inside GraduateStudent, use the **@Override** annotation tag, and change the logic check to >= 95.0.

```
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

// — PARENT CLASS BLUEPRINT (Fully Encapsulated) —
class Student {
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    private String name;
    private int[] grades;

    // Automated Parent Constructor
    public Student(String rawName, Scanner inputScanner) {
        this.name = formatName(rawName);
        this.grades = new int[SUBJECTS.length];
        fillGradesArray(inputScanner);
    }

    // Public Getters
    public String getName() {
        return this.name;
    }

    public int[] getGrades() {
        return this.grades;
    }

    // Baseline Honor Roll Rule (90.0% or higher)
    public boolean qualifiesForHonorRoll() {
        return calculateAverage() >= 90.0;
    }

    // Private Instance Helpers
    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    private void fillGradesArray(Scanner inputScanner) {
        System.out.println("\n--- Entering Grades for Class Schedule ---");
        for (int i = 0; i < this.grades.length; i++) {
            System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
            String textGrade = inputScanner.next();

            try {
                this.grades[i] = Integer.parseInt(textGrade);
            } catch (NumberFormatException e) {
                System.out.println("✗ Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
                this.grades[i] = 0;
            }
        }
    }

    public double calculateAverage() {
        int sum = 0;
        for (int score : this.grades) {
            sum += score;
        }
        return (double) sum / this.grades.length;
    }
}

```